

Bounded Analysis of the `scrontab` Master-Runner Consolidation in `generate_workflows.sh`

NOAA EIB — Global Workflow PR Test Harness Review

April 30, 2026

Abstract

This note evaluates the change introduced on branch `copilot/modify-scrontab-scripts` of the `global-workflow.DavidH` fork, which collapses N per-experiment `scrontab` entries into a single sequential master-runner job. We frame the change in terms of the only function the `cron/scron` entry performs — driving the Rocoto workflow tick — and place bounded estimates on its memory, CPU, and throughput consequences. The conclusion is that the absolute resource saving is well under one percent of either the `scron` service node or the downstream PR-sweep compute footprint, but is materially useful in the narrow case of a small `cgroup` on the `scron` job (the actual Gaea failure mode).

1 Scope and System Context

The CM for `global-workflow` maintains an out-of-band PR test harness driven by `dev/workflow/generate_workflows.sh`. The script generates an experiment per PR YAML case under `dev/ci/cases/pr/`, and registers a scheduled tick that advances each experiment’s Rocoto workflow.

The scheduled tick has exactly one purpose: invoke `rocotorun` (or the experiment’s `*.scron.sh` wrapper) so that Rocoto reads its XML, queries/updates the per-experiment SQLite workflow database, and submits whatever cycle tasks have become eligible to Slurm. The tick performs no science, holds no compute resources, and is independent of every actual model job that Rocoto subsequently submits to the batch system.

The branch under review changes *only* how the tick is scheduled on hosts that use Slurm-managed `scrontab` (Gaea). All downstream behaviour — per-experiment XML, SQLite database, `EXPDIR/COMROOT/DATAROOT`, task graph, and resource footprint — is unchanged.

2 Pre- and Post-Change Behaviour

Before. For each experiment, the per-experiment crontab block (`$pslot.crontab`, containing its `#SCRON --*` directives and a `* /5 * * * *` line invoking `$pslot.scron.sh`) was appended to `tests.cron`. Installation via `scrontab` created N independent `scron` jobs — one per experiment — each firing every five minutes and each launching its own `rocotorun` on the `scron` service node concurrently.

After. The script collects each experiment’s `*.scron.sh` path into `_scron_sh_files`, then writes a single `rocoto_master_run.sh` under `$RUNTESTS/EXPDIR/` which calls each `scron` script sequentially. A single `master_scron` block is emitted to `tests.cron` reusing the `#SCRON --partition=/--account=`

directives drawn from the first experiment’s crontab and adding `#SCRON --dependency=singleton`, ensuring the next 5-minute tick cannot start until the previous one drains.

The transformation is purely a concurrency-flattening of the tick work: total CPU-seconds and total I/O are unchanged; only the concurrency profile differs.

3 Bounded Cost Model

Let N be the number of PR cases, $T = 300$ s the tick period, R the resident set size of one `rocotorun` pass, and t its wallclock duration. Plausible ranges drawn from typical Rocoto deployments and the GW PR XML sizes are summarised in Table 1.

Table 1: Bounding parameters per `rocotorun` invocation.

Quantity	Plausible range	Source of bound
RSS of one <code>rocotorun</code> (R)	50–300 MB	Ruby + XML DOM + libsqlite + <code>squeue</code> fork
Wallclock of one pass (t)	2–30 s	Dominated by <code>squeue/sacct</code> round-trip + XML parse
CPU-time per pass	$\approx t$	Single-threaded Ruby
PR cases (N)	≈ 10	Current sweep size; up to ~ 30 plausible
Tick period (T)	300 s	Hardcoded <code>*/5 * * * *</code>

3.1 Peak Concurrent Footprint on the `scron` Node

Before — N parallel ticks.

$$M_{\text{peak}} = N \cdot R \in [10 \cdot 50, 10 \cdot 300] \text{ MB} = \mathbf{0.5\text{--}3 \text{ GB}} \quad (1)$$

$$C_{\text{peak}} = N \text{ cores busy for } t \text{ s} \in [20, 300] \text{ CPU-s per 300-s window} \quad (2)$$

After — one serial tick.

$$M_{\text{peak}} = R \in \mathbf{0.05\text{--}0.3 \text{ GB}} \quad (3)$$

$$C_{\text{peak}} = 1 \text{ core busy for } N \cdot t \text{ s} \quad (4)$$

Total CPU-seconds and total scheduler RPC count are identical.

3.2 Duty Cycle of the Master Tick

$$D = \frac{N \cdot t}{T} \in \left[\frac{10 \cdot 2}{300}, \frac{10 \cdot 30}{300} \right] = [0.7\%, 100\%] \quad (5)$$

The lower end is comfortably idle. The upper end — long passes combined with many cases — is precisely where the `singleton` dependency starts *stretching* the per-experiment polling cadence and the implicit throughput cost becomes real. Realistic GW PR sweeps sit near $D \approx 5\text{--}15\%$, so the nominal 5-minute cadence is preserved.

Table 2: Magnitude comparison (typical service-node and PR-sweep scales).

Reference	Order of magnitude
Memory saved (this PR), peak	$\sim 0.5\text{--}3\text{ GB}$
Memory saved (this PR), average	$\sim 50\text{--}300\text{ MB}$
Typical scron-node total RAM (Gaea es-login)	256–500 GB
Saved fraction of node memory	$< 1\%$
One C384 forecast job memory footprint	hundreds of GB across many compute nodes
One PR sweep’s compute-node-hours	$10^3\text{--}10^4$ core-hours
Master-tick CPU work saved	0 (same total, just serialised)

4 What the Saving Is Worth

Comparing the saving against the things that actually share the service node and against the PR sweep’s downstream footprint:

So as a fraction of either (a) the scron service node or (b) the resources the PR sweep itself consumes downstream on compute nodes, the change sits in the noise floor — well under 1%.

4.1 Where the Bound Is Not Negligible

The one regime where the change is meaningful is when the `cgroup` applied to the scron job is intentionally small:

- If the cron partition allocates each scron job e.g. **1 core / 2 GB** (a common defensible default), then $M_{\text{peak}} = 0.5\text{--}3\text{ GB}$ *straddles or exceeds the limit*, causing OOM-kill of the tick. This is almost certainly the failure mode the Gaea CM observed; the PR fixes it cleanly.
- If the cgroup is generous ($\geq 8\text{ GB}$, $\geq 4\text{ cores}$), the change is essentially cosmetic.

4.2 Aggregate Site-Wide Impact

If every active GW developer simultaneously ran a 10-case PR sweep (say ~ 5 concurrent users), the aggregate difference in Slurm controller load is on the order of:

$$\Delta \approx \underbrace{5}_{\text{users}} \cdot \underbrace{10}_{\text{exp/user}} \cdot \frac{1 \text{ burst}}{300 \text{ s}} \approx \mathbf{0.17 \text{ rpc/s}} \quad (6)$$

against a controller that comfortably handles $10^3\text{--}10^4 \text{ rpc/s}$. Invisible.

5 Throughput Trade-off

The actual model jobs’ wallclock is unchanged — Rocoto still submits them to Slurm exactly as before. What changes is the cadence at which each experiment’s task graph is re-evaluated:

- **Before:** every experiment’s `rocotorun` fires every 5 min in parallel; per-experiment polling cadence is constant in N .

- **After:** the master walks experiments serially within a single 5-min tick. If the cumulative pass time ($N \cdot t$) stays well below T , cadence is preserved. If it does not, `singleton` blocks the next tick and per-experiment cadence stretches roughly to $N \cdot t/T \cdot 5$ min.

This is bounded but real: it scales linearly in N and in `rocotorun`’s pass time. For current PR loads the effect is sub-second to sub-minute and immaterial; at higher case counts or with large XMLs it would degrade end-to-end PR-sweep wallclock by adding idle time between a task completing and the next dependent task being submitted. The current implementation provides no mitigation (e.g. capping the master at < 5 min, or sharding into 2–3 master jobs).

6 Recommendations

1. **Reframe the rationale.** The accurate one-line description is: *“prevent OOM-kill of the scron-managed Rocoto tick on Gaea by serialising per-experiment passes,”* not an aggregate efficiency improvement.
2. **Validate the cgroup assumption** on Gaea (and any other scron-using site) so the trade-off is documented and site-portable.
3. **Per-site partition/account.** The master block reuses the partition/account of the *first* experiment’s crontab; this is fine when all PR cases target the same service partition (typical) but would silently misroute the driver tick if a future case ever specified a different one. Consider asserting equality across collected `*.crontab` files before emitting the master block.
4. **Bound the master pass.** Add a guard inside `rocoto_master_run.sh` so that if the cumulative wallclock approaches T , remaining experiments are deferred to the next tick rather than relying solely on `--dependency=singleton` to absorb the overrun.

7 Conclusion

The branch correctly addresses a real, site-specific operational failure on Gaea — OOM-kill of the cron-driven Rocoto tick under a constrained `cgroup` — using a clean, contained change to a single CM utility. Its framing as a memory or efficiency improvement overstates the system-level impact: as a fraction of either the scron node or the PR sweep’s compute footprint, the saving is below 1%, and it introduces a bounded, $O(N)$ throughput cost in worst-case polling cadence. The trade is favourable for the present PR sweep size; it would warrant revisiting if PR case counts or `rocotorun` pass time grow materially.